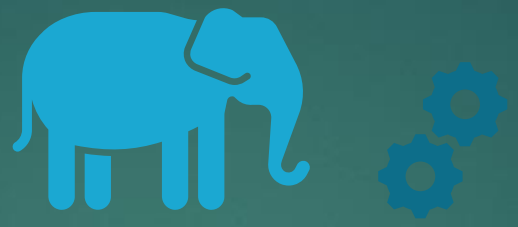


3. Gradle build tool

LEIM – DAM

What is Gradle?

- Gradle is an **open-source build automation tool** used to **compile, test, package, and deploy software projects**

- Built on JVM, supports Java, Kotlin, Groovy, Scala, C/C++
- Uses a Groovy (build.gradle) or Kotlin (build.gradle.kts) DSL instead of XML
- Designed for multi-project builds
- Highly customizable and extensible
- Used by Google (Android), Netflix, LinkedIn, Spring, …

DSL - Domain-Specific Language : is a language designed for a particular task, with a particular syntax and grammar

Why Do We Need a Build Tool?

- Without a build tool, developers must manually:
  - Compile source files
  - Manage dependencies
  - Run tests
  - Package artifacts
  - Deploy applications
- A build tool automates these tasks

- Typical automated pipeline:

Source code
↓
Compile
↓
Run tests
↓
Packaging (JAR / APK)
↓
Publish / Deploy

Gradle allows defining this pipeline **as tasks**

Key Features

Incremental Builds

Only recompiles what changed, saving significant build time

Build Cache

Reuses outputs from previous builds, even across machines

Parallel Execution

Runs independent tasks in parallel for faster builds

Build Lifecycle

1

Initialization

Determines which projects are part of the build

2

Configuration

Configures all project objects and creates task graph

3

Execution

Runs the selected tasks in the correct order

Groovy & Kotlin DSL

Groovy DSL - build.gradle

```
apply plugin: 'java'

repositories {
    mavenCentral()
}

dependencies {
    implementation 'com.google:guava:31.1'
}
```

Kotlin DSL - build.gradle.kts

```
plugins {
    java
}
repositories {
    mavenCentral()
}
dependencies {
    implementation("com.google:guava:31.1")
}
```

Gradle allows build configurations to be written using a **Domain-Specific Language (DSL)** based on either:

- Groovy - traditional Gradle syntax (build.gradle)
- Kotlin - type-safe Kotlin syntax (build.gradle.kts)

These DSLs replace traditional XML configuration used in older build tools such as Apache Maven or **Apache Ant,** allowing developers to write **more expressive and programmable build scripts** with the use of variables, functions, loops, conditionals, …

Groovy versus Kotlin DSL

Feature	Groovy DSL (.gradle)	Kotlin DSL (.kts)
Performance	Faster first-time configuration.	Faster subsequent builds (better caching).
Syntax	Flexible, "loose," and concise.	Strict, type-safe, and explicit.
Learning Curve	Easy for quick scripts.	Slightly steeper if you don't know Kotlin.
Standard	Legacy/Maintenance mode.	**Current Android Default.**

Dependency Management

- **Dependency configurations: implementation, api, compileOnly, runtimeOnly, testImplementation**
  - Define how and when dependencies are visible and used during compilation, runtime, and testing phases.
- **Transitive dependency resolution with conflict strategies**
  - Gradle automatically resolves dependencies required by other dependencies and applies strategies to handle version conflicts.
- **Supports Maven Central, JCenter, Google, and custom repositories**
  - Gradle can download libraries from public repositories or from private repositories defined by the project.
- **Dependency locking for reproducible builds**
  - Locks exact dependency versions so that future builds use the same resolved versions.
- **BOM (Bill of Materials) support for version alignment**
  - Allows multiple related dependencies to use compatible versions defined in a central BOM.
- **Rich version constraints: strictly, require, prefer, reject**
  - Provides fine-grained control over dependency versions to enforce compatibility and avoid problematic versions.

Gradle Wrapper

- Ensures consistent Gradle version across all environments
- No need to install Gradle manually
- Files included in version control:
  - gradlew/gradlew.bat (scripts)
  - gradle/wrapper/**gradle-wrapper.properties**
  - gradle/wrapper/gradle-wrapper.jar
- Usage: ./gradlew build instead of gradle build
- Upgrade: ./gradlew wrapper --gradle-version 8.5

Gradle Wrapper

- **Gradle Wrapper — No Need to Install Gradle Manually**
- The Gradle Wrapper allows a project to **use Gradle without requiring developers to install Gradle on their machine**.
- **How it works**
- The project includes small wrapper files:
  - gradlew (Linux/macOS script)
  - gradlew.bat (Windows script)
  - gradle/wrapper/gradle-wrapper.jar
  - gradle/wrapper/gradle-wrapper.properties
- When running: *./gradlew build* the wrapper:
  - **checks which Gradle version the project requires**
  - **downloads that version automatically** (if not already cached)
  - **runs the build using that exact version**
- **Advantages**
  - No manual Gradle installation required
  - Every developer uses the **same Gradle version**
  - Builds are **more reproducible and consistent**
  - Easier setup for new developers and CI systems

Summary

The Gradle Wrapper ensures that a project automatically downloads and uses the correct Gradle version, eliminating the need for developers to install Gradle manually.

Gradle Project Structure

- Example structure of a Gradle project:

```
project-root/
│
├── build.gradle.kts
├── settings.gradle.kts
├── gradlew
├── gradlew.bat
├── gradle/
│   └── wrapper/
│
└── app/
    ├── build.gradle.kts
    └── src/
        ├── main/
        │   ├── java/
        │   └── resources/
        └── test/
```

- Important files:

File	Purpose
settings.gradle.kts	defines included projects
build.gradle.kts	build configuration
gradlew	Gradle wrapper
gradle/wrapper	wrapper binaries

Gradle Project Structure

- **Root Level (The Orchestrators)**
  - These files manage the project as a whole and ensure everyone on your team is using the same version of Gradle.
  - **settings.gradle.kts**: This is the very first file Gradle looks at. It defines the project name and tells Gradle which modules (like the /app folder) to include in the build.
  - **build.gradle.kts (Root)**: This is for configuration that applies to the entire project. In modern Android development, this is where you usually define the **plugins** (like the Android Gradle Plugin or Kotlin) that will be used across all modules.
  - **gradlew & gradlew.bat**: These are the **Gradle Wrapper** scripts (shell for macOS/Linux, batch for Windows). **Always use these** instead of a local Gradle installation. Running ./gradlew build ensures the project downloads and uses the specific Gradle version it was designed for, preventing "it works on my machine" bugs.
  - **gradle/wrapper/**: This folder contains the .jar and properties file that the wrapper scripts use to actually download the correct Gradle distribution.

Gradle Project Structure

- **App Level (The Logic)**
  - This is where your actual Android application lives.
  - **app/build.gradle.kts**: This is the most important file for your day-to-day work. It contains the specific configuration for your app module, including:
    - compileSdk and targetSdk versions.
    - **Build Types** (debug, release).
    - **Dependencies** (libraries like Retrofit, Room, or Compose).
  - **app/src/**: The source container.
    - **main/**: Your production code. java/ (or kotlin/) holds the logic, and resources/ holds things like XML layouts, strings, and images.
    - **Test/**: This is for **Unit Tests** that run on your computer's JVM (not on a device).

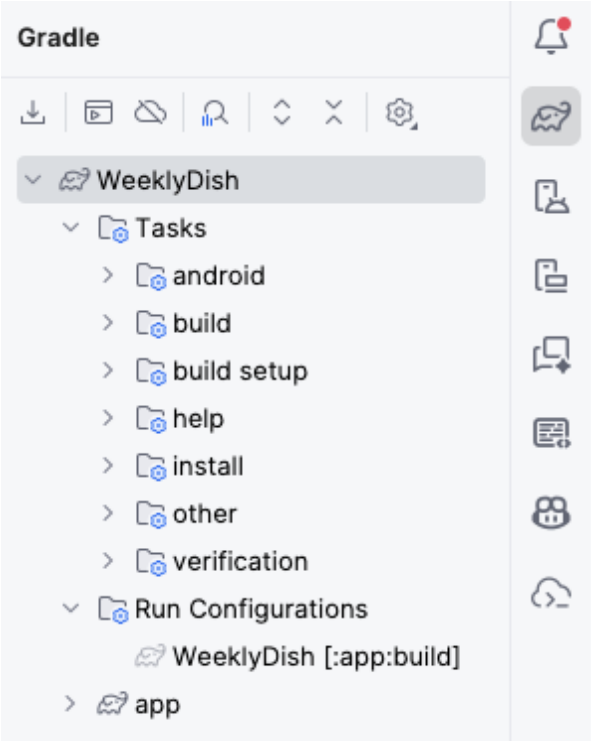
Gradle Project Structure

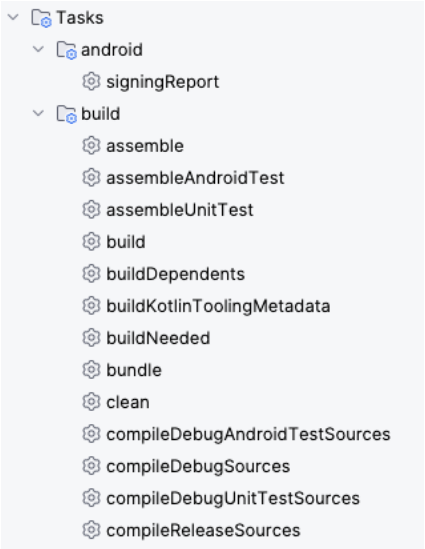
File/Folder	Function
settings.gradle.kts	Defines project structure and modules.
build.gradle.kts (Root)	Project-wide plugin and repository settings.
gradlew	The "magic" script to run Gradle without installing it.
app/build.gradle.kts	Your app's specific recipe: SDKs, dependencies, and build types.
src/main/	The heart of your app (Code + UI resources).

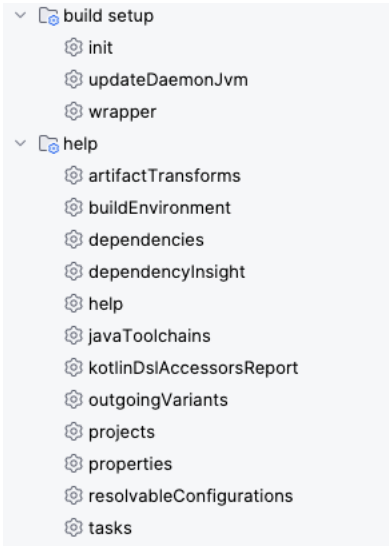
Gradle Tasks

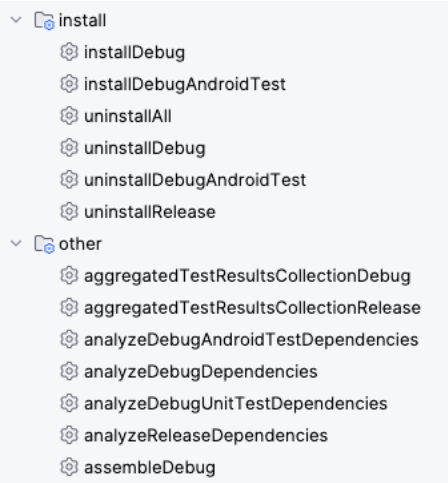
- A **task** is the smallest unit of work in Gradle.
- Examples:
  - compileJava
  - test
  - jar
  - build
  - clean
- Run tasks from command line:
  - ./gradlew build
  - ./gradlew test
  - ./gradlew clean
- Info:
  - ./gradlew tasks
  - ./gradlew dependencies
  - ./gradlew help
- Tasks can depend on other tasks

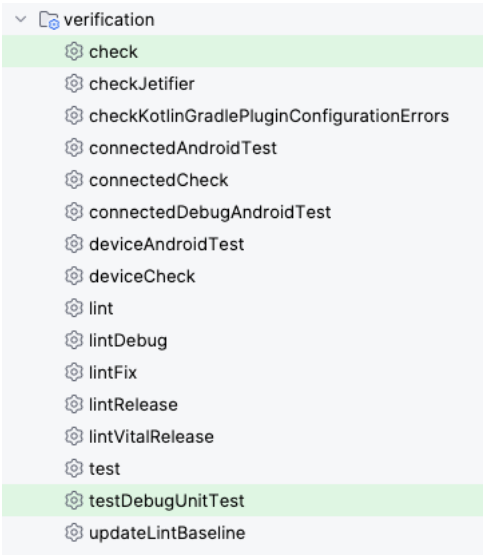
Gradle tasks – example











Task Dependency Graph

- Gradle builds a **task graph** before executing the build.
- Example:

```
build
├── test
│   └── compileTestJava
└── jar
    └── compileJava
```

- Gradle automatically determines:
  - execution order
  - parallelization opportunities
  - skipped tasks (incremental builds)

Creating Custom Tasks

- Gradle allows creating custom tasks.
- Example in **Kotlin DSL**:

```
tasks.register("hello") {
    doLast {
        println("Hello Gradle!")
    }
}
```

- Run it with:

```
./gradlew hello
```

- Tasks can automate:
  - code generation
  - documentation
  - deployment
  - testing pipelines

Multi-Project Builds

- Gradle is designed for **large modular systems**.

- Example:

```
root
├── settings.gradle.kts
├── app
├── core
├── network
└── database
```

app, **core**, **network**, and **database** are **Gradle modules (subprojects).**
A **module** is an **independent component of the project** that has its own: source code, build.gradle / build.gradle.kts, dependencies, compilation process.
Modules help **organize large systems into smaller and reusable parts**.

- settings.gradle.kts:

```
rootProject.name = "MyProject"
include("app")
include("core")
include("network")
include("database")
```

- Benefits:
  - modular architecture
  - faster builds
  - reusable components

Modules - example

- **Module responsibilities**
  - **app** – Main Android application (Activities, UI, navigation, APK/AAB generation)
  - **core** – Shared utilities, base classes, common models used across modules
  - **network** – API communication, HTTP client configuration, remote services
  - **database** – Local persistence (Room entities, DAO classes, database setup)
  - **feature-login** – Feature module implementing the login functionality (new module)

- **Library projects:** In Gradle, a project does **not need to produce an executable application**. It may simply provide **reusable (modules) libraries or components**.
  - Base project defines shared modules: **core**, **network**, **database**.
  - Mobile app project declares **app** (mobile UI application) and uses shared modules
  - B2B platform project declares **app** (web or enterprise application) and uses shared modules

Modules

- Working with **modules** in Gradle provides the following architectural and productivity advantages, especially in medium and large projects.
  - **Better project organization:** Modules divide a large system into **smaller, well-defined components**, making the codebase easier to understand and maintain.
  - **Separation of concerns:** Different functionalities are isolated into dedicated modules.
  - **Code reuse:** Modules can be reused across multiple projects or applications.
  - **Faster builds:** Gradle can **build only the modules that changed**, instead of rebuilding the entire project. This improves **incremental build performance**.
  - **Parallel compilation**: Gradle can **compile modules in parallel**, reducing total build time in large systems.
  - **Easier team collaboration**: Different teams or developers can work on **different modules independently**, reducing merge conflicts
  - **Clear dependency management:** Modules declare their dependencies explicitly.
  - **Independent testing:** Modules can have independent unit tests, allowing developers to test components in isolation

Gradle in Android Development

- Android Projects use:
  - Android Gradle Plugin (AGP)
- Example plugin:

```
plugins {
    id("com.android.application")  // Android Gradle Plugin
    kotlin("android").             // Kotlin support for Android

}
```

- AGP manages:
  - processing resources
  - dependency resolution
  - build variants
  - APK / AAB generation (**APK** - Android Package Kit; **AAB** - Android App Bundle)

Build Variants/types

- Android Gradle supports **build types**
  - A build type is a different configurations of the same codebase
- Example:
  - debug
  - release
- Configuration example:

```
android {
    buildTypes {
        release {
            isMinifyEnabled = true
        }
    }
}
```

- Gradle generates different builds automatically.

Build Variants - example

```
android {
     buildTypes {

          getByName("debug") {
               applicationIdSuffix = ".debug"
               isDebuggable = true
          }

          create("staging") {
               initWith(getByName("debug")) // Inherit debug settings
               applicationIdSuffix = ".sta"
               buildConfigField("String", "API_URL", "\"https://staging.api.com\"")
          }

          getByName("release") {
               isMinifyEnabled = true
               proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
          }
     }
}
```

debug and **release** already exist by default and can be configured. **staging** does not exist by default, so it must be created.

Dependencies

- **Dependencies** are **external libraries or modules that a project needs in order to compile and run.** Instead of writing everything from scratch, developers reuse existing libraries that provide useful functionality such as networking, JSON parsing, logging, testing frameworks, UI components, …
- Gradle automatically downloads and caches dependencies.
- Gradle resolves dependencies through repositories:

```
repositories {
    mavenCentral()
    google()
}
```

- Dependency example:

```
dependencies {
    implementation("org.jetbrains.kotlin:kotlin-stdlib") // Kotlin Standard Library
    implementation("com.company.platform:core:1.0.0")    // dependency from core module
}
```

What is a repository?

- A **repository** is an **online storage location that hosts software libraries** (artifacts) that developers can reuse in their projects.
- These repositories contain:
  - compiled libraries (.jar, .aar)
  - metadata
  - version information
  - dependency information
- When Gradle builds a project, it **automatically downloads the required libraries from these repositories**.

"Supports Maven Central, JCenter, Google, and custom repositories"

Supported repositories

- **Supported repositories include:**
  - Maven Central – The primary public repository for Java and JVM libraries, widely used across the ecosystem. The essential home for Android-specific libraries like Jetpack, Compose, and the Android Gradle Plugin (AGP).
  - Google Maven Repository – Provides Android and Google libraries such as AndroidX and Google Play Services.
  - JCenter – Formerly popular repository for Java libraries *(now deprecated and no longer recommended)*.
  - **Custom repositories** – Developer-defined repositories (private or internal) used to host proprietary or organization-specific libraries.

Gradle Build Performance

- Gradle improves build performance using:
  - **Incremental compilation**
    - Only changed files are recompiled.
  - **Build cache**
    - Reuse build outputs across builds.
  - **Parallel execution**
    - Multiple tasks run simultaneously.
  - Command:

```
./gradlew build --parallel
```

Gradle vs Maven (as build tools)

Feature	Gradle	Maven
Build Script	Groovy / Kotlin DSL	XML (pom.xml)
Performance	Faster (incremental + cache)	Slower (full rebuilds)
Flexibility	Highly customizable	Convention over configuration
Learning Curve	Steeper	More straightforward
Android Support	Official build tool	Limited support

Gradle is preferred for **Android and large modular systems.**

When Should You Use Gradle?

- Gradle is ideal when:
  - building **Android applications**
  - working with **multi-module systems**
  - automating **complex pipelines**
  - managing **large dependency graphs**
  - improving **build performance**

When Should You Use Maven?

- Apache Maven is ideal when:
  - **building standard Java projects** with a well-defined and conventional structure
  - **prioritising simplicity and predictability** over flexibility
  - **working in teams that prefer convention over configuration**
  - **using large enterprise ecosystems** where Maven is the established standard
  - **integrating with mature CI/CD tools** that have strong Maven support
  - **maintaining legacy or long-lived Java projects** already built around Maven

- 💡 Summary
  - **Gradle** → better for **flexible**, **complex**, or **performance-sensitive** builds
  - **Maven** → better for **stable**, **standardized**, and **convention-driven projects**

What Happens When You Run gradlew build?

1. Gradle Wrapper starts
2. Gradle loads settings.gradle
3. Projects are initialized
4. build.gradle files are configured
5. Task graph is created
6. Tasks are executed
7. Outputs are produced